

Operating Systems

202.1.3031

Spring 2026 Assignment 2

Synchronization



Ben-Gurion University
of the Negev

Contents

1	Introduction	2
2	Submission Instructions	3
3	Task 0: Pseudo-Random Number Generator	4
4	Task 1: Israeli Lock	6
5	Task 2: Relay Race Tournament	12

1 Introduction

Welcome back to another exciting chapter in the world of operating systems! We hope that by now you feel more comfortable working with xv6 and are ready to explore synchronization from a slightly less conventional angle. In this assignment, you will implement a new locking mechanism in xv6: the Israeli lock. This lock is inspired by the Israeli queue, where social connections can sometimes influence who gets served next. In our version, processes that “know each other”—that is, belong to the same group—may receive preferential treatment when competing for a lock. As you might expect, this can improve access for some, while raising interesting questions about fairness for everyone else.

To build this mechanism, you will implement a small pseudo-random number generator, extend xv6 with group identities for processes, and then design a lock that blends ordinary FIFO discipline with a configurable amount of “favoritism”. Along the way, you will gain hands-on experience with process synchronization, kernel extensions, and the tradeoff between fairness and efficiency. You will also have a chance to experiment with different levels of “favoritism” and observe how even a simple change in policy can significantly affect the behavior of the system.

An operating system, even a small and elegant one like xv6, is a complex piece of software. Low-level code can still be frustrating, subtle, and at times surprisingly stubborn. That is completely normal. These things take time, but they can also be fun, satisfying, and occasionally even funny. We hope this assignment helps connect ideas from operating systems, concurrency, and system design in a way that is both instructive and memorable. Take a deep breath, be patient with yourself, and when things do not work, keep calm and start debugging!

2 Submission Instructions

- Make sure your code compiles without errors and warnings and runs properly!
- We recommend that you comment your code and explain your choices, if needed. This would also be helpful for the discussion with the graders.
- In-order to avoid errors caused by changes made in the previous assignment it is recommended to start this assignment with a fresh clone of xv6.
- Submit a **single copy** of xv6 containing all of the changes you made for all tasks, in a single `.tar.gz` or `.zip` file. All tasks must be completed in the same copy of xv6 without breaking the operating system or the other tasks.
- We advise you to work with git and commit your changes regularly. This will make it easier for you to track your progress, collaborate and maintain a working version of the code to compare to when things go wrong.
- Theoretical questions in this assignment are **not for submission**, but you are advised to understand the answers to these questions for the grading sessions.
- Submission is allowed **only in pairs** and **only via Moodle**. Email submissions will not be accepted.
- Before submitting, run the following command in your xv6 directory:

```
make clean
```

This will remove all compiled files as well as the `obj` directory.

- Help with the assignment and git will be given during office hours. Please email us in advance.

3 Task 0: Pseudo-Random Number Generator

We start with a warm-up. In this assignment, we will later assign each process to a group and use randomness to determine lock transition. Unfortunately, true randomness is not available on deterministic machines like classical computers. Therefore, we use pseudo-randomness. In a typical system, one might use the pseudo-random generator with the `srand` and `rand` functions from the standard library, but such functions are not available in xv6. Therefore, we begin by implementing a simple pseudo-random number generator.

A *pseudo-random number generator* (PRNG) is a deterministic algorithm that, given a seed (an initial value), produces a sequence of numbers that appear random.

There are many types of PRNGs; the one we will implement is a **linear congruential generator** (LCG). An LCG is defined by three integer parameters (a, b, m) and generates values according to:

$$x_{n+1} = a \cdot x_n + b \pmod{m}$$

We will implement two functions in the kernel, corresponding to a simplified version of the standard `srand` and `rand` functions. You should define a global (static) variable that stores the current state x_n . *Note: This variable can be changed by many processes at once and would need a synchronization mechanism for short periods of time. Which simple xv6 lock suits this need?* The function `srand` initializes this state using a seed, and `rand` advances the generator by one step and returns the new value.

Important Note

LCGs are not secure PRNGs. Do not use them for security-critical purposes (e.g. cryptography).

Task 0 — Pseudo-random Number Generator

1. Define a global randomness state variable.
2. Implement the two functions in the kernel:
 - (a) `void lcg_srand(uint seed);`
 - (b) `uint lcg_rand(void);`
(use the parameters ($a = 1664525, b = 1013904223, m = 2^{32}$)).as described above. Make sure there are no synchronization problems when attempting to read and write to the randomness state variable.
3. Add a system call for each one of these functions making them also accessible from userspace.

For submission: Add the function signatures to `user.h` and the implementation to `ulib.c`.

4 Task 1: Israeli Lock

In this task, you will implement a new synchronization mechanism in xv6 called the **Israeli lock**. Unlike standard locks that may or may not enforce strict fairness (depending on the the design of the lock), this lock explicitly introduces a controlled amount of unfairness in the form of *favoritism* between processes.

The idea is inspired by the notion of an **Israeli queue**, in which individuals may advance in line if they know someone already waiting. In our setting, processes that belong to the same group may receive preferential access to the lock. This behavior is governed by a parameter called the **favoritism coefficient**, which controls how likely it is for such favoritism to occur.

Synchronization between processes in xv6 requires kernel support, since processes cannot directly access each other's memory. You will therefore implement the Israeli lock as a kernel-level mechanism exposed through system calls.

Process Groups

To model “social connections”, each process will be assigned a group identifier (*gid*). Two processes are considered *connected* if they share the same *gid*.

Add a field to the process structure:

```
struct proc {  
    ...  
    int gid;  
};
```

Implement the following system calls:

1. `void setgid(int gid);`
Sets the group identifier of the calling process.

2. `int getgid(void);`
Returns the group identifier of the calling process.

Each process should assign itself to a group using the PRNG implemented in Task 0:

```
setgid(lcg_rand() % groups);
```

where `groups` is a parameter controlling the number of groups.

Lock Design

The Israeli lock maintains a queue of waiting processes. When the lock is released, the next process to acquire it is chosen according to the following policy. Let G be the `gid` of the process releasing the lock and let c be the favoritism coefficient (in percent, between 0 and 100). Then:

1. Among waiting processes, identify those with `gid == G`
2. If such processes exist:
 - With probability $c\%$, select one of them (the earliest among them in the queue).
 - Otherwise, select the next process in FIFO order.
3. If no such processes exist:
 - Select the next process in FIFO order.

This mechanism preserves a base FIFO structure while allowing controlled deviations from fairness.

We define the following system calls:

1. `int israeli_create(int favoritism);`
Creates a new Israeli lock with the given favoritism coefficient (0–100). Returns a unique identifier for the lock. In case the lock cannot be created, returns `-1`.

2. `int israeli_acquire(int lock_id);`
Attempts to acquire the lock. If the lock is already held, the calling process should yield the CPU and try again later. It must not busy-wait. When the function returns, it is guaranteed that the calling process is holding the lock. Returns 0 on success and `-1` on error.
3. `int israeli_release(int lock_id);`
Releases the lock and selects the next owner according to the policy described above. Returns 0 on success and `-1` on error.
4. `int israeli_destroy(int lock_id);`
Deletes the lock with the given identifier. Once this function returns, the lock identifier becomes invalid. Returns 0 on success and `-1` on error.

Implementation Notes

1. The data structure for the Israeli lock will be maintained in the kernel. Design a structure that supports the required fields, namely to keep track of:
 - whether the lock is active
 - the queue of waiting processes
 - the favoritism coefficient
2. When checking if favoritism should take place, remember the PRNG functions you've implemented in Task 0.
3. Take inspiration from the implementation of `spinlock` and `sleeplock` in the kernel.
4. The lock must maintain a FIFO queue of waiting processes. Favoritism may only be applied to processes already in the queue. No process may bypass the queue entirely.
5. For simplicity, you may assume that each Israeli lock will be used by at most 16 processes. Therefore, you are not required to implement a dynamic queue. Instead, you may use a fixed-size array to represent the queue of waiting processes.
6. When a process fails to acquire the lock, it should either call `yield()` and retry later, or use the `sleep` and `wakeup` mechanism of xv6. Either way, the implementation must not busy-wait.

7. Instruction reordering and memory visibility may cause incorrect behavior. You may use:

- `__sync_lock_test_and_set`
- `__sync_lock_release`
- `__sync_synchronize`

to enforce atomicity and memory ordering where appropriate.

8. Next, create a global array of at least 15 Israeli locks and initialize it when the kernel starts. See the initialization of the `proc` array for an example.

9. Finally, test your implementation using a userspace program with multiple processes competing for the lock. For example, you may use the following test program:

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#define NPROC 15
#define GROUPS 4

int main(void) {
    int lock_id = israeli_create(50);
    lcg_srand(getpid());

    for (int i = 0; i < NPROC; i++) {
        int pid = fork();

        if (pid == 0) {
            int gid = lcg_rand() % GROUPS;
            setgid(gid);

            israeli_acquire(lock_id);

            printf("Process %d (gid=%d) acquired the lock\n",
                  getpid(), getgid());

            sleep(10);
            israeli_release(lock_id);
            exit(0);
        }
    }
}
```

```
    for (int i = 0; i < NPROC; i++)  
        wait(0);  
  
    israeli_destroy(lock_id);  
    exit(0);  
}
```

Task 1 — Israeli Lock Implementation

1. Add a gid field to struct proc.
2. Implement the system calls:
 - (a) void setgid(int gid);
 - (b) int getgid(void);
3. Add the Israeli lock data structure to the kernel and initialize an array of at least 15 locks.
4. Implement the four system calls:
 - (a) int israeli_create(int favoritism);
 - (b) int israeli_acquire(int lock_id);
 - (c) int israeli_release(int lock_id);
 - (d) int israeli_destroy(int lock_id);
5. Test your implementation with a userspace program using multiple processes. You can use the following example code for testing. Try to change the values of the number of groups (GROUPS) and the favoritism coefficient and see how it affects the results.

For submission: Changes to the xv6 kernel to implement the lock mechanism. You do not have to submit the test code.

Questions — not for submission

1. What is the purpose of the Israeli lock? How does it differ from a standard FIFO lock?
2. How does the favoritism coefficient affect system behavior?
3. Can starvation occur in this system? If so, under what conditions?
4. Why do we maintain a queue of waiting processes? What would go wrong without it?
5. Why do we prefer processes to yield the CPU or sleep instead of busy-waiting?
6. What are the key fields required in the data structure for the Israeli lock?
7. How do we ensure that the lock is held by only one process at a time?
8. Why are atomic operations and memory barriers needed in this implementation?
9. How would the system behave if all processes had the same `gid`?
10. How would the system behave if each process had a unique `gid`?
11. Does `yield()` guarantee that another waiting process will run next?
12. How does this lock relate to real-world scheduling or priority mechanisms?

5 Task 2: Relay Race Tournament

In this task, you will use the Israeli lock implemented in Task 1 to simulate a relay race between several competing teams.

The goal of this task is not only to verify that your lock works correctly, but also to observe how synchronization policy can influence the outcome of a concurrent system. In particular, we will examine how favoritism between processes of the same group may affect fairness and performance.

The Story

Imagine a relay race with several teams of runners. Each runner is represented by a process. A runner may advance the team only when holding the shared baton.

There is only one baton in the race. Therefore, at any given moment, only one runner may hold it. In our implementation, the baton is represented by the Israeli lock.

Acquiring the lock corresponds to taking the baton. Releasing the lock corresponds to passing the baton so another runner may continue.

Processes belonging to the same team should share the same group identifier (`gid`). Thus, the favoritism mechanism of the Israeli lock models the idea that runners from the same team may receive the baton more often.

Requirements

Write a userspace program that creates several teams of runners and simulates the relay race.

Your program should satisfy the following requirements:

1. Create a single Israeli lock using:

```
int lock_id = israeli_create(c);
```

where *c* is the favoritism coefficient.

2. Create multiple child processes using `fork()`.
3. Divide the child processes into teams. For example:
 - 3 teams of 5 runners each
 - or 5 teams of 3 runners each
4. Assign each process a team identifier by calling:

```
setgid(team_id);
```
5. Maintain a shared score for each team.
6. Each runner should repeatedly perform the following loop:
 - (a) Acquire the Israeli lock
 - (b) Increase the score of its team by 1
 - (c) Print a short message including:
 - process id
 - team id
 - updated score
 - (d) Release the lock
 - (e) Sleep briefly
7. The race ends when one of the teams reaches a target score.

You may begin with 3 teams, 5 runners per team and a target score of 30. Then experiment with different values.

Important Note: Since different processes must update the same team scores, you will need a mechanism to keep track of them. You may choose to implement it as you see fit. Recommended: implement the scores inside the kernel.

Example Output

```
Runner 12 (Team 1) acquired the baton
```

```
Team 1 score = 8
```

```
Runner 14 (Team 1) acquired the baton
```

```
Team 1 score = 9
```

```
Runner 7 (Team 0) acquired the baton
```

```
Team 0 score = 6
```

Run your program with different values of the favoritism coefficient. For example, $c = 0$ (pure FIFO), $c = 50$, and $c = 100$ (maximum favoritism)

Observe the following:

1. Which team wins the race?
2. Do same-team streaks occur?
3. Are the final scores balanced?
4. Does one team dominate the race?
5. Does higher favoritism improve throughput?
6. Does higher favoritism reduce fairness?

Task 2 — Relay Race Tournament

1. Write the relay race simulation described above.
2. Simulate at least three different values of the favoritism coefficient.
3. Record the final scores and the winning team in each run.
4. See that you understand and can answer the questions below.

For submission: Your userspace relay race program `relay_race.c` and any kernel modifications you made.

Questions — not for submission

1. How did the favoritism coefficient affect the winner of the race?
2. Did runners from the same team tend to acquire the baton consecutively?
3. Which configuration produced the fairest race?
4. Which configuration produced the least fair race?
5. Can a team fall permanently behind? Why?
6. How does this experiment relate to scheduling policies in real systems?